Thesis Title

Michael Reape
20750531

Final Year Project – 2025
B.Sc. Single Honours in
Computer Science and Software Engineering

Department of Computer Science
Maynooth University
Maynooth, Co. Kildare
Ireland

A thesis submitted in partial fulfilment of the requirements for the B.Sc. Single Honours in Computer Science.

Supervisor: **Mark McCormack**

Contents

List of Figures

List of Tables

Declaration

I hereby certify that this material, which I now submit for assessment on the program of study as part of **Bachelor of Science** qualification, is *entirely* my own work and has not been taken from the work of others - save and to the extent that such work has been cited and acknowledged within the text of my work.

I hereby acknowledge and accept that this thesis may be distributed to future final year students, as an example of the standard expected of final year projects.

Signed:  Date: 16/03/2025

Acknowledgements

I would like to thank my Mother, Geraldine and her partner Gerry for their help and support getting me this far.

Abstract

A memory palace is a powerful tool for enhancing recall but its application relies on a strong ability for visualising real life locations and mnemonic devices. Previous attempts at creating a virtual equivalent have all had the same issue of an inventory limiting the imagination of the user. This project aims to address this by providing the user with the ability to create any sort of mnemonic device they can perceive using AI-generated imagery and 3D objects. The application will provide an intuitively designed, scalable virtual environment where the user can place and store their memory aids. An effective save and load system will be implemented for seamless retrieval of the memory palace. The environment will be navigated from a first-person perspective providing a realistic point of view of the images and objects created. The project was evaluated on its scalability, the performance of the AI models and the reliability of virtual environment. This system provides a cutting edge alternative to traditional memory recall tools. Further work could explore VR implementation for added immersion as well as optimisation of the AI models for greater performance.

Chapter 1: Introduction

1.1 Motivation

The memory palace or Method of Loci is a technique widely used to aid memorisation and recall. It involves visualising an environment, usually a familiar space, and placing mnemonic devices throughout that environment. These devices serve as memory cues, helping recall information when mentally navigating the space. However this method relies on a strong ability to visualise and recall the spatial details, which may be challenging for some individuals.

To adapt this technique to a digital medium, several applications have attempted to provide virtual memory palaces, but are often limited by a fixed inventory of mnemonic devices. This restriction reduces the flexibility and personalisation needed for effective memory training. An effective virtual memory palace should allow users to generate and place custom mnemonic devices, adapting them to their own memory needs.

This project aims to address this limitation by integrating a Stable Diffusion AI model with a customisable virtual environment. The application will enable the user to

- Create a personalised virtual environment.
- Generate 2D and 3D mnemonic devices from a text prompt.
- Save and reload their memory palace, preserving the object placement.

1.2 Problem Statement

To achieve this, an AI model capable of generating relevant 3D objects must be integrated into an interactive virtual space. The system should allow users to customize the environment to fit their needs. It should allow images and object to be generated from within the environment and placed in the space as well as have multiple memory palaces saved and reloaded when needed.

The main technical challenges involved will be

- Building a system to create scalable virtual environments.
- Selecting and integrating the AI model to generate the mnemonic devices
- Developing an API layer to allow communication between the virtual environment and AI model.

- Implementing a lightweight save/load system to store and reconstruct the user-created environments.

1.3 Approach

To build the application the following steps were taken:

1. Building the Virtual Environment
    - Unity was chosen as the game engine
    - To give the user control over the size of the environment the Wave Function Collapse algorithm was chosen to generate the map, Eliminating the need to construct predefined map layouts.
    - A set of unique room prefabs were constructed in Blender and imported as assets to Unity
2. Selecting the AI model
    - Various AI models were researched to match the requirements of being free to use for research, locally hostable and produce an object in a reasonable amount of time
3. Developing the API
    - Since AI models typically run on Python a Flask based API framework was chosen and implemented
    - The API was tested with Unity using local files first then the AI models integrated
4. Implementing the Save/Load functionality
    - Saving the entire 3D environment is resource-intensive, so a lightweight system was designed to rebuild the environment using the Wave Function Collapse algorithm
    - The Map and Object data is serialised and stored in a JSON file which reuses the instantiation methods to rebuild the map when loaded providing an efficient solution.

1.4 Metrics

The application will be evaluated on

Scalability: How efficiently it generates and loads the environments of various sizes

AI Performance: Time required to generate the mnemonic devices

User Experience: The ease of generating, placing, saving and loading the mnemonic devices and the reliability of the save/load functionality.

1.5 Significant Achievements

1. Implementing a procedurally generated virtual environment using the Wave Function Collapse algorithm
2. Integrating a Stable Diffusion AI model into the Unity application via a Flask API
3. Enabling real time mnemonic device instantiation in Unity based on user provided text prompts
4. Developing a lightweight save/load system for storing and retrieving user-created memory palaces.

Chapter 2: Technical Background

2.1 Topic material

In analysis of mnemonic devices Gordon Bower provides a scientific breakdown of why the Method of Loci is effective. The study found that structured spatial cues significantly improved memory recall by as much as two to seven times compared to rote memorisation. The process relies on associating an image or object that represents the information with a location(loci), resulting in easier recall when traversing the memory palace mentally.

While visual imagery plays an important role in the technique, the paper suggests that the images do not have to be highly detailed or realistic to improve recall. More important is the placement and organisation of the mnemonic devices. The study also showed that training is required to maximise the benefits of the Method of Loci as strong visualisation of the environment and object is crucial for better performance.

More recent studies have examined the effectiveness of the Method of Loci in virtual environments. Research published in Acta Psychologica compared a conventional Method of Loci approach to one using a virtual environment. The findings showed no significant difference in recall between traditional and virtual methods, despite the participants using a space they were very familiar with in the traditional method and only seeing the virtual space for the first time. Both techniques outperformed the control group, demonstrating the effectiveness of the technique in memory enhancement. Notably, participants who used a first-person navigation system within the virtual environment exhibited substantially improved recall compared to the control group and found it much easier to use the technique compared to the those who mentally traversed a familiar space. Additionally, the study found no significant difference between high-detail and low-detail virtual environments, suggesting that intricate textures or models may not be necessary for effective use of the technique. High-imageability words were found to be easier to recall in both studies, hinting at the importance of a strong visual representation for the information to be remembered. The study also noted that traditionally users of the technique have used their own homes for the familiar mental environment meaning that people with larger homes could have an advantage over those who may live in a one-bedroom apartment for example, the use of a virtual environment negates this.

An important consideration in the study was that participants were only allowed a maximum of five minutes to explore the virtual environment, possibly longer exposure to the environment may further increase recall ability.

2.2 Technical material

To implement a virtual memory palace, several technologies were researched and incorporated into the project. These technologies include Unity to build the application and virtual environment, procedural map generation through the Wave Function Collapse algorithm, image generating AI models that use Stable Diffusion and an API framework for communication between Unity and AI model.

Unity (reference) was selected as the platform for developing the virtual environment for its powerful 3D rendering capabilities, extensibility, and support for first-person interaction. The engine provides essential tools for real-time rendering, API communication and user interaction, making it an ideal choice for constructing a dynamic and immersive memory palace.

Unity's GLTFast plugin(reference) was researched to be used to instantiate the objects into the virtual environment.

For procedural map generation, the Wave Function Collapse algorithm was explored as a solution to dynamically construct the memory palace environment. Originally developed by Maxim Gumin, the algorithm is inspired by quantum mechanics and uses pattern-based constraints to generate structures.

The algorithm begins with a grid where each cell starts in a superposition state, meaning it can be any tile from a predefined set. A cell is randomly selected and collapsed into an observed state, after which adjacency constraints propagate the changes throughout the grid. This ensures that the resulting structure remains connected and follows logical spatial rules. The WFC approach eliminates the need for manually pre-designed maps, allowing for flexible and scalable virtual environments.

Stable Diffusion (reference)was investigated as a means of generating mnemonic images and 3D objects from user-provided text prompts. Popular models were researched for suitability on Hugging Face, making use of its Diffusers(reference) library for simple implementation.

An API was necessary to enable communication between Unity and the AI inference backend. Various frameworks were considered, but Flask was chosen due to its lightweight nature, ease of deployment. As it is a Python framework it offers seamless compatibility with the AI backend.

Chapter 3: The Problem

3.1 Project UML documentation

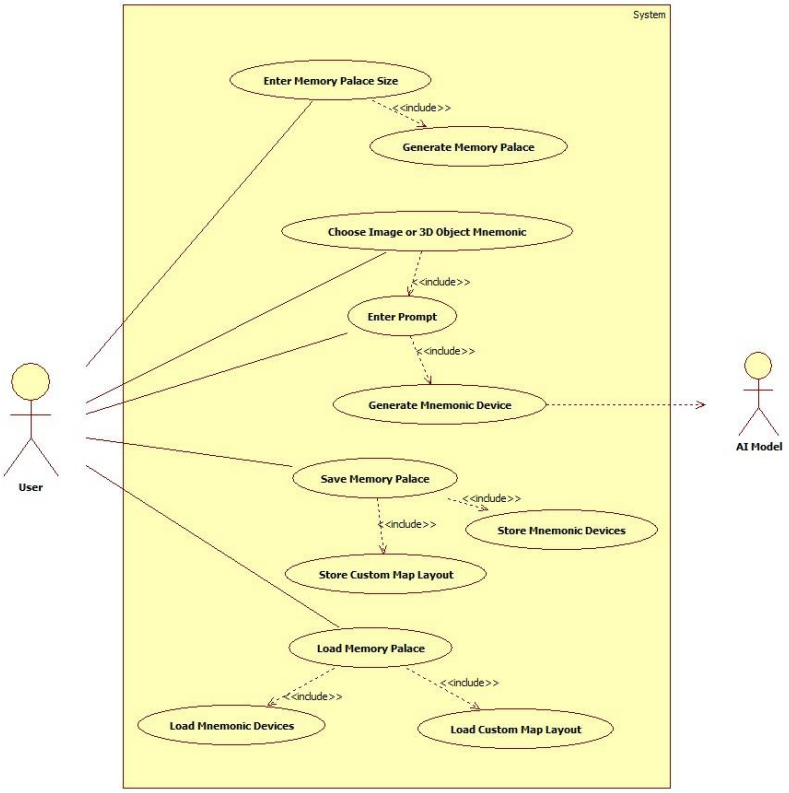


Figure 3-1 UML use case diagram

3.2 Requirements

Functional Requirements

- The system must allow the user to create a virtual memory palace of a designated size
- The system must provide a UI for choosing which type of device is requested and for entering a text prompt.
- The system must communicate with the AI model through an API to send the prompt and receive the mnemonic device.
- The system must support the real time instantiation of the mnemonic devices received from the AI model.
- The User must be able to save and load the virtual memory palace, including the placement of the mnemonic devices.

Non-Functional Requirements

- Scalability: The virtual environment should be able to handle large memory palaces, both number of rooms and devices
- Performance: Mnemonic devices should be generated within a reasonable time of sending the prompt

- Usability: Interface must be simple, allowing the user to seamlessly generate a mnemonic device
- Reliability: Save/Load system should accurately store and reconstruct the virtual memory palace.

Chapter 4: The Solution

4.1 Architectural Level

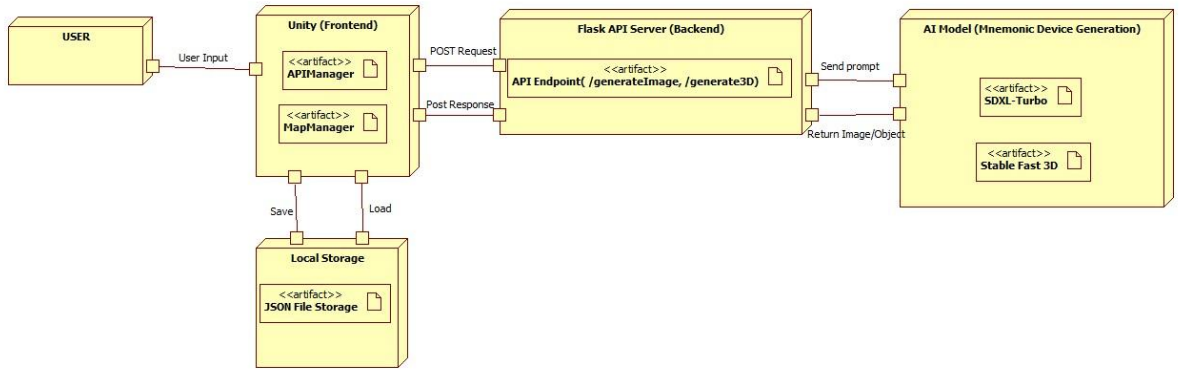


Figure 4-1 Architecture diagram

4.2 High Level

4.2.1 Unity virtual environment:

The virtual environment is a procedurally generated map made up of room prefabs connected together. These prefabs were modelled in Blender and imported to Unity. The map is generated using a Wave Function Collapse algorithm which allows the users to specify the size of the map using a grid based input.

For example a 2x2 grid will create a map of four connected rooms and a 3x3 grid will create a map of nine connected rooms.

This approach was chosen as it

- Allows the user to specify the size of environment they require
- Can create unique and varied maps automatically
- Offers scalability with creation of larger maps from a smaller, limited number of room prefabs

Each room contains four mnemonic device “spawners”, one in each corner. Images can be displayed on a canvas above each spawner. Objects can be placed freely within the open space of the room.

This simple layout was chosen as it

- Gives the user creative freedom to place objects throughout the space
- Also aligns with the method of Loci principle, encouraging the user to position the mnemonic devices in a distinct uncrowded location for better recall.

The user navigates the environment from a first person perspective. Movement is controlled with the WASD keys and interaction with the left mouse click. Users can generate, place and interact with the 3D objects dynamically.

The first-person perspective was chosen

- To give an immersive experience to the user
- To mimic real world perspective in memory techniques

4.2.2 AI Models:

SDXL-Turbo from Stability AI was selected for generating mnemonic images based on user prompts. The model processes a text based input and outputs a 512 x 512 pixel image using stable diffusion.

This is also used as a preprocessing step for generating the 3D objects, as its output serves as the input to the Stable Fast 3D model.

This model was chosen because it

- Is popular and well documented model
- Had lower VRAM requirements than other models tested, such as Stable Diffusion 3.5 Medium. More efficient for local hosting.
- Has a fast inference speed, producing an image in 30 seconds when tested
- Is free to use for research purposes

Stable Fast 3D was chosen, also from Stability AI, for generating the 3D mnemonic objects. This model takes a 512 x 512 pixel image in PNG format as input and produces a UV unwrapped textured 3D mesh.

This model was chosen because it

- Is well documented and easily accessible through Hugging Face and GitHub for simple implementation.
- Has compatibility with the chosen image generation model, its input aligning with SDXL-Turbo's output, minimising parsing.
- Has reasonable generation speed, producing an object in 4 minutes when tested on an older CPU, Ryzen 3600.
- Is free to use for research purposes

4.2.3 Flask API:

The Flask API serves as the communication layer between the Unity virtual environment and the AI models. It receives prompts from Unity via HTTP requests, processes them, and forwards them to the appropriate AI model. Once the model generates an output the API sends the result back to Unity for instantiation

Flask was chosen as it's

- Lightweight and easy to integrate
- Widely used and well documented
- Python based like the AI implementation

4.2.4 Save/Load System:

The save/load system lets the user store and reload their custom virtual environments. It preserves the image and object data after saving including their location, the objects scale and rotation. The data is saved in a structured JSON format.

JSON-based storage was chosen due to its

- Simple serialisation of a data structure, MapData class
- Human readability

4.3 Low Level

4.3.1 Wave Function Collapse Algorithm

The Wave Function Collapse is a procedural generation algorithm used to create the virtual environment from a series of tiles (room prefabs). A grid of cells is created and initialised so that each cell has the same entropy or number of possible tiles they can be.

The process works as follows

- A cell with the lowest entropy (fewest possible tile choices) is chosen and collapsed. If multiple cells have the same entropy then one is chosen.
- The collapsed cells choice is propagated to its neighbours in the grid, reducing their entropy by removing tiles that violate the adjacency constraints.
- This process is repeated until all cells have been collapsed.

1. Grid Initialization

The InitialiseGrid method sets up the grid structure used for procedural generation. It takes height and width as parameters, creating a 2D array of GridCell objects

GridCell class contains:

- Grid Position (Vector2Int): The cell's coordinates in the grid.
- Chosen Tile (null initially): Stores the assigned tile once collapsed.
- Spawner Image Path (string): Holds an image reference for object placement (explored later).
- Possible Tiles (List): Contains all valid tiles for that cell.

TileData class contains:

- Tile Name (string): Room type identifier.
- Door Positions (Binary List): Represents doors on south, west, north, and east walls.
- Prefab Reference (GameObject): Unity prefab reference for the tile.

2. Remove Boundary Tiles

After initialisation, the grid is passed to the RemoveBoundaryTiles method. This step ensures that edge and corner cells only allow valid tiles by eliminating rooms with outward facing doors.

3. Get Lowest Entropy

RunWFC starts a loop that will run until all cells have been collapsed. Within the loop GetLowestEntropy is called, which uses a linear search minimum selection algorithm to find the lowest entropy cell in the grid. It performs a linear search (O(n)), which is efficient for moderate grid sizes.

4. Collapse Cell

The Lowest entropy cell is collapsed by selecting a random valid tile, leaving the cell in an observed state.

5. Propagate

The observed cells new adjacency constraints are propagated to its immediate neighbours using a vector offset. Each adjacent cell must have a door that corresponds to the collapsed cell's layout. For example,

if the collapsed cell is a north tile, it will only have one door facing north and so the only compatible tiles for a cell to the north of it's position will have a south facing door.

6. Check Collapsed

The current grid is iterated through to verify if all the cells have been assigned a chosen tile. If any cell remains in super-position, the algorithm returns to step 3 and repeats. Otherwise, it proceeds to verify a path.

7. Check Path

To ensure every room is accessible, a Breadth First Search algorithm is used to verify valid paths between rooms.

1. The (0,0) cell is added to the queue and marked visited.
2. The first cell in the queue is dequeued.
3. Its door positions are checked.
4. For each valid neighbouring cell, if doors correctly align, it is added to the queue.
5. This process continues until the queue is empty.
6. If the number of visited cells matches the grid size, return true and continue to step 8.
7. Otherwise, return false and return to step 2.

8. Instantiate Tiles

The list of cells is iterated through and the room prefab corresponding to the chosen tile instantiated into the scene creating the virtual environment. The mnemonic device spawners are initialised with a grid location and index.

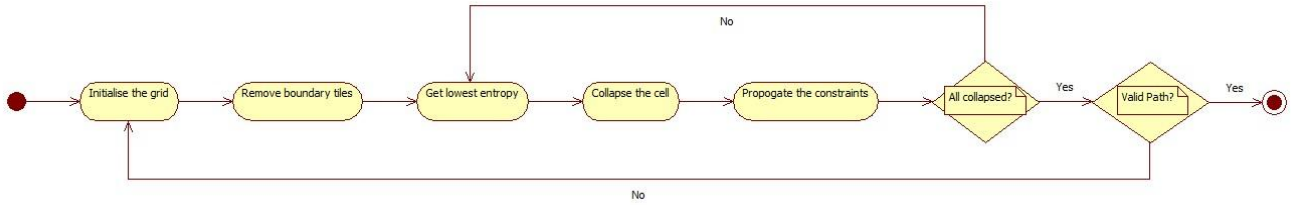


Figure 4-2 WFC Activity diagram

4.3.2 API Communication

The APIManager class handles the communication between Unity and the AI models. A prompt is provided and sent to the appropriate API endpoint using UnityWebRequest. The API upon recieving the request, processes it and sends the prompt to the appropriate model.

The User selects what type of mnemonic device to generate, an image or a 3D object. They enter a prompt and confirm. Depending on which type selected either GetImageFromAPI() or Get3DObjectFromAPI() is called in the APIManager class. If an image is requested, along with the text prompt the method recieves additional parameters, the rooms co-ordinates in the grid and the spawners index. These parameters are used to store and reconstruct the map later, ensuring the images are kept in the same place.

A coroutine is then started to handle the async request, a form created and the prompt embedded as part of the request body. This is sent as a POST request using UnityWebRequest.

The Flask API handles this request running on port 5000, providing two endpoints

/generateImage

- The API extracts the text prompt from the request body.
- The prompt is passed to the SDXL-Turbo model, which generates an image.
- The image is saved in a byte stream format.
- The image is returned to Unity as a PNG file.

/generate3D

- The API extracts the text prompt from the request body.
- The prompt is first passed to the SDXL-Turbo model, which generates an image.
- This image is saved as a temporary PNG file.
- The temporary image is then sent to the Stable Fast 3D model, which processes it and generates a GLB file.
- The path to the GLB file is returned to the API which returns the file to Unity as a GLB file.
- The temporary files are removed.

4.3.3 Image & Object Instantiation

Once the mnemonic devices have been generated and returned to Unity they must be instantiated into the scene.
The image data is received as a byte array and loaded into a Texture2d object. This texture is converted to a sprite and passed to controller it was spawned from. Attached to this controller is an image GameObject, the sprite is applied to this and appears in the scene.
For the 3D object a GLB file is returned to unity and is immediately written to storage. The APIManager returns the file path of the object to the controller. The loadObject class is passed this file path and makes use of the GLTFast plugin to instantiate it into the scene as a GameObject. A box collider is attached to the object to facilitate interaction and the ObjectFuncitonality class applied to it enabling it to be dragged, rotated and scaled. The object is spawned above the user's head and can then be positioned around the scene.

4.3.4 Save Feature

The MapData class is a data structure designed to hold the information of the images, objects and rooms necessary for recreating the map from a JSON file.

Each room in the map has

- An x and y position, which cell in the grid array it belongs to
- A roomType, the name of the room prefab that fills the cell
- An array to hold the path in the persistent data storage to the image that belongs to the spawners in the room

Each object in the map has

- A name, the full file name that it is saved with in the storage, eg object_1.glb
- Its position, rotation and scale

Both of these are stored in a RoomData List and ObjectData List respectively, along with the maps width, height and cellsize.

When the user saves the map from the pause menu, a method in the MapManager saveMap is called. This compiles all this data into the MapData object and sends it to the MapSaver class to be written to storage as a JSON.

The format of the JSON string is {width, height, cellsize, RoomData, ObjectData}

An interesting point on saving the object, when an object is dragged its position, scale or rotation may change. Upon release updateData method is called which updates this information in the running object data List. If the object is deleted it will be removed from this list. When the map is saved again these changes are written to the JSON file.

4.3.5 Load Feature

To load a map from the JSON file it is read and the map reconstructed from the data.

- The map is selected from a scroll view list of all the maps in the save directory
- Once selected, the LoadMap method is called from the MapLoader class.
- A new MapData object created by deserialising the data using Unity's JsonUtility
- The Map is passed to the ReconstructGrid method in the MapManager class. A new grid is created from the dimension provided and each cell iterated through assigning its room to the corresponding roomType in the loaded MapData object.
- The image paths are set to each cell
- The instantiateTiles method called, reusing the same method when creating a new map.
- To load the 3D objects into the loaded map the gltFast plugin is used, the object data from the json is passed to an asynchronous method LoadObjectFromSave.
- The object is instantiated and its position, scale and rotation set from the object data.

Chapter 5: Evaluation

Summary

This chapter evaluates the performance and effectiveness of the application based on the previously mention metrics of scalability, AI performance and ease of user experience.

5.1 Solution Verification

The evaluation will examine how well the system generates the virtual environment, the time required for the mnemonic devices to be instantiated into the scene after a prompt and how accurately the memory palace is reproduced after saving.

5.2 Scalability

5.2.1 Maximum Reliable Environment Size

The system was tested by generating progressively larger environments and its performance was analysed with a focus on consistency. A result of 95% was required to be considered consistent. The test was run by restricting the number of time the WFC algorithm could retry generating a map with a valid path to 20000, the number of times needed was printed to the console when the map generated. A failure was recorded if an error occurred in the program.

Grid size	Success rate	Avg. retries when successful
5 x 5 (25 rooms)	10/10	6
6 x 6 (36 rooms)	10/10	120
7 x 7 (49 rooms)	10/10	720
8 x 8(64 rooms)	10/10	1458
9 x 9 (81 rooms)	8/10	3458

Figure 5-1 Scalability test results table

5.3 AI Performance

5.3.1 Mnemonic Device Generation Time

The AI model was tested to find the average time taken to instantiate an mnemonic device after a prompt was submitted. Both the image and 3D object mnemonics were tested repeatedly with a prompt and time recorded manually. The system these models are running on is obviously a large factor in the results. Both were tested on a Ryzen 3600 CPU.

- Image generation averaged 36 seconds.
- 3D object generation averaged 6 minutes 30 seconds.

5.4 Data Persistence

5.4.1 Accuracy of Memory Palace Reconstruction

The save/load functionality of the system was tested to ensure that all images, objects and room placement was fully restored each time a map was loaded

- 100% reliability was found with no missing images, objects or incorrectly placed room prefabs.
- All sizes of map that were successfully constructed the first time and saved were fully restored.

5.5 Interpretation of Results

- The application effectively and consistently generates a 64 room virtual environment without issue.
- Image instantiation time is acceptable, even on a low-mid range system.
- 3D object instantiation time is slow and effects the usability of the application.
- Save/Load functionality is robust even with larger environments.

Chapter 6: Conclusion.

6.1 Contribution to the state-of-the-art

The integration of AI-generated mnemonic devices with a scalable virtual environment was a novel approach to solving a traditional problem associated with this memory tool. Previous systems have constrained their users with a predefined inventory. The approach taken in this project not only permitted the opportunity for more creativity to the user but also less work for the developer, removing the need to laboriously create a vast inventory of items.

6.2 Results discussion

The system being capable of consistently and efficiently creating a virtual environment of 64 rooms, each of which containing four image spawner and the potential to store many more 3D objects means this should in no way hinder the user. The ability to accurately and reliably save and reconstruct the entire memory palace further expands the scale of the system, enabling multiple memory palaces if 64 rooms proved insufficient.

The AI performance results are a limiting factor in the effectiveness of the system, particularly the 3D object generation. Assuming all users of the application would not have access to high-end Nvidia GPUs for potentially much faster inference times, a project based solely around 2D mnemonic devices may have more practicality.

6.3 Project Approach

The project was developed using Unity as the tool for developing the interactive virtual environment and proved very capable, providing useful tools and plugins such as UnityWebRequest and GLTFast.

The use of Flask as the API framework was also practical and provided a seamless integration with the AI models, providing an effective solution for managing the API requests that were crucial to this project.

The decision to use a JSON based save/load system was simple and efficient, allowing consistent and reliable storage and retrieval of the data.

6.4 Future Work

While the project did successfully implement a scalable virtual environment with AI technology, several areas could be improved and explored further:

1. Performance optimisation of the AI models for faster processing times.
2. If faster processing times were not possible the idea of sequential or queued prompts to the AI model could be investigated.
3. Virtual reality integration would further enhance the immersion and realism of the experience, potentially leading to greater recall ability for the user.
4. Although as revealed in the topic research that a realistic environment was not necessary for effective recall, a more sophisticated tile set could lead to greater customisation and immersion for the user.
5. The efficiency of the save feature to store so much information in such a lightweight solution could make it compatible with a share feature where users could potentially collaborate with one another.

References

Action Research : A definition . (2015). Retrieved February 25, 2016, from http://valenciacollege.edu/faculty/development/tla/actionResearch/ARP_softchalk/ARP_softchalk_print.html

Hammersley, M. (1993). On the teacher as researcher. In M. Hammersley (Ed.), *Educational Research: Volume One: Current Issues* (pp. 211–231). The Open University.

Jick, T. D. (1979). Mixing Qualitative and Quantitative Methods: Triangulation in Action. *Source: Administrative Science Quarterly Qualitative Methodology*, *24*(4), 602–611. Retrieved from http://www.jstor.org/stable/2392366

Kemmis, S. (1993). Action Research. In M. Hammersley (Ed.), *Educational Research: Volume One: Current Issues* (pp. 175–190). The Open University.

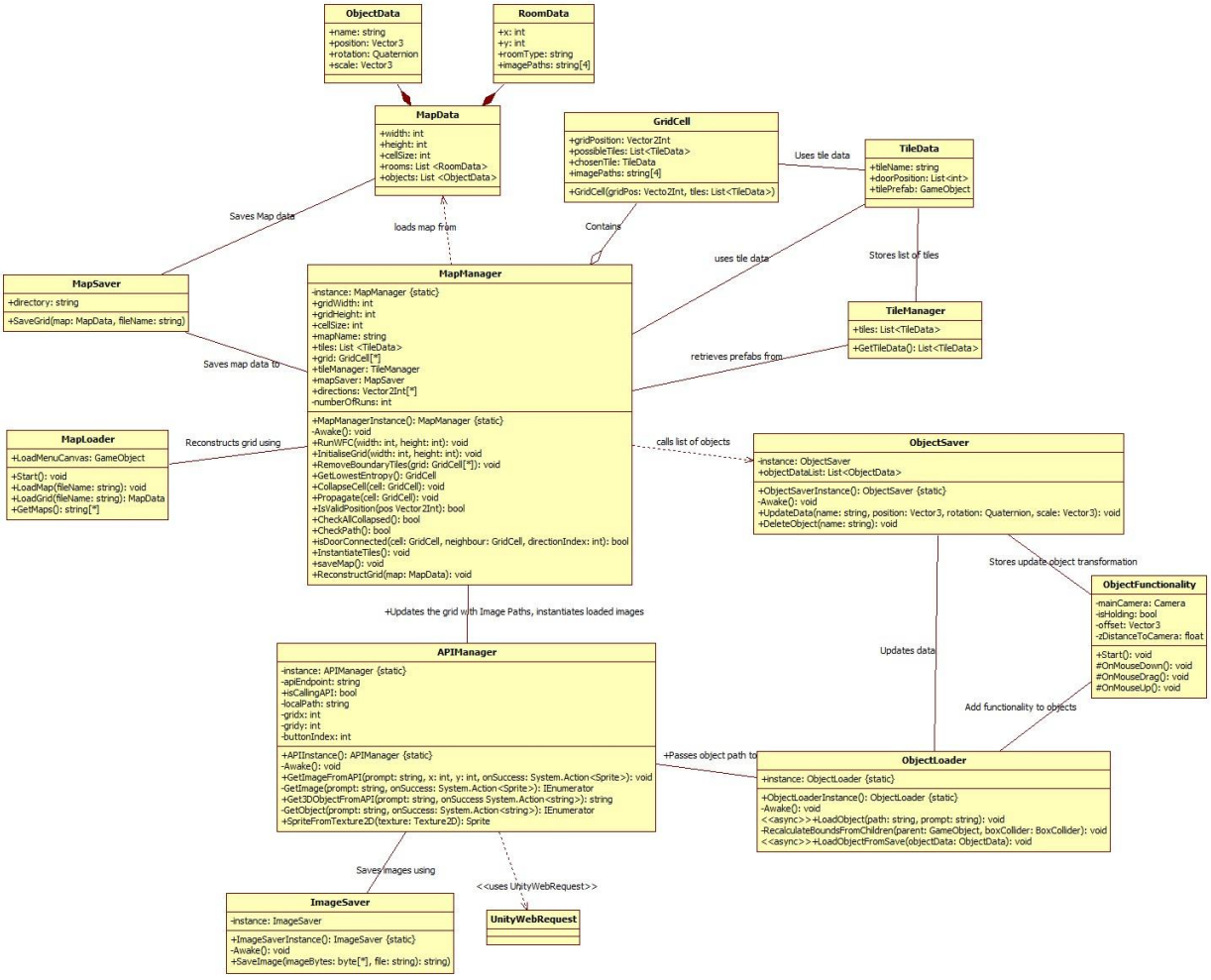
Kolb, D. (1984). *Experiential learning*. New Jersey: Prentice Hall.

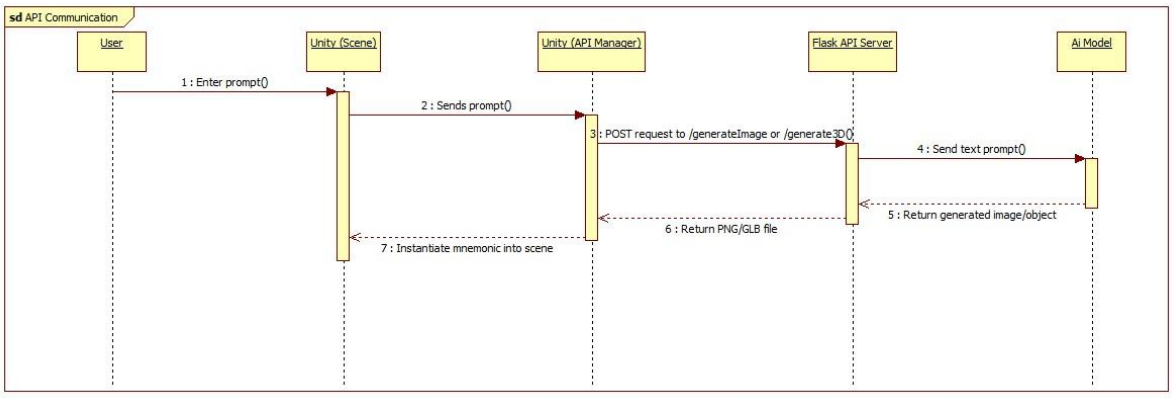
McNiff, J., Lomax, P., & Whitehead, J. (2003). *You and Your Action Research Project* (2nd ed.). London & New York: London & New York.

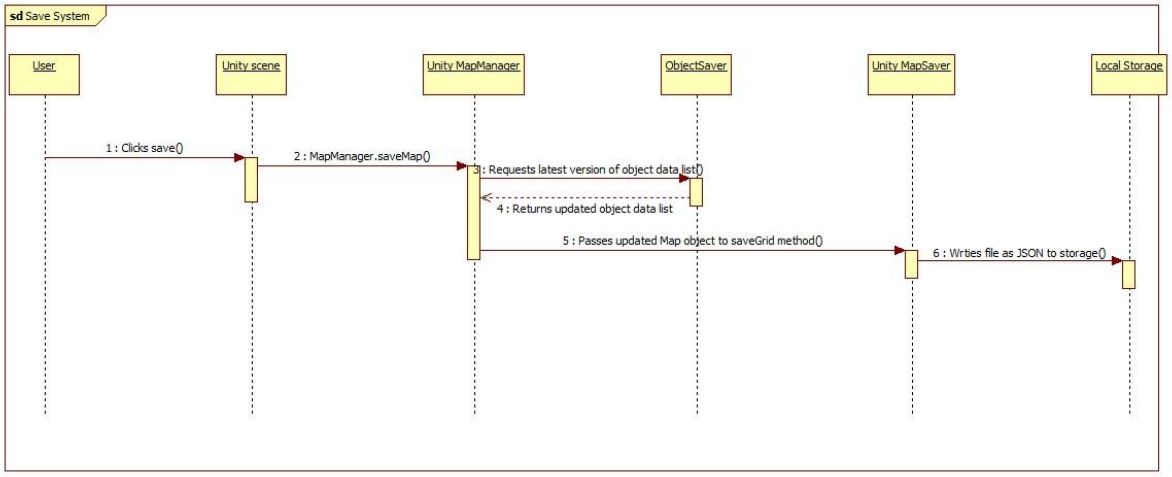
Ma, Mengmeng, Jian Ren, Long Zhao, Davide Testuggine, and Xi Peng. "Are multimodal transformers robust to missing modality?." In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 18177-18186. 2022.

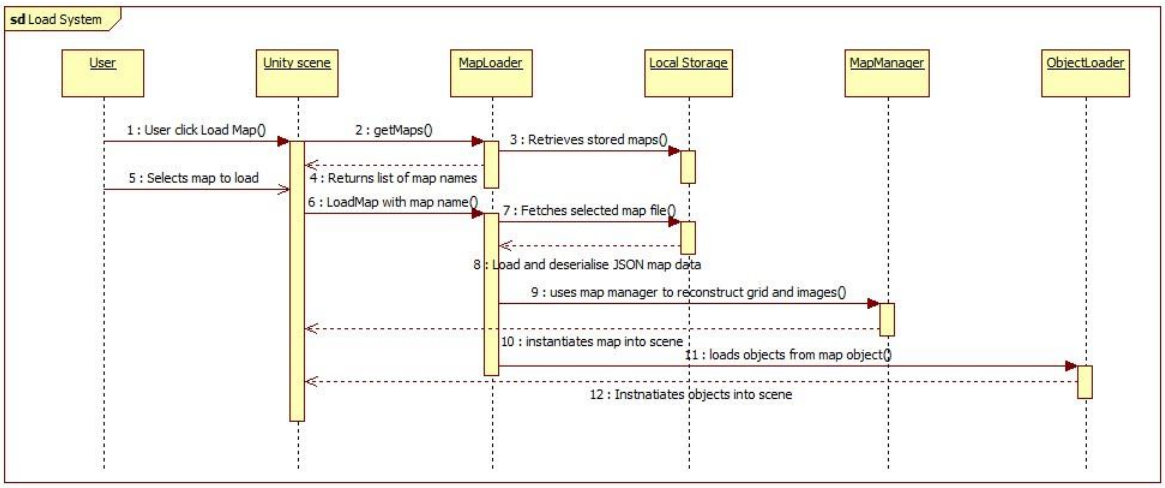
Appendices

Appendix 1 UML Class, Use Case and sequence diagrams for this project.









Appendix 2 Evidence Supporting use of LLM (GhatGPT, GitHub Copilot etc)

2.1 Prompts and responses.

2.2 In-line completions.

Appendix 3 Code developed for this project.

GitHub for submission: https://github.com/MarkMcCormackMU/STAMPEDE

GitHub used throughout development: https://github.com/MichaelReape/projectStampede

Links are clickable for easier navigation.

- WFC scripts
  - MapManager.cs
  - GridCell.cs
  - TileData.cs
  - NewMap.cs
  - MapSize.cs
- Spawner scripts
  - APIManager.cs
  - ButtonController.cs
  - ObjectLoader.cs
  - PrompCanvasController.cs
- Save-Load scripts
  - ImageSaver.cs
  - LoadMenuController.cs
  - MapData.cs
  - MapLoader.cs
  - MapSaver.cs
  - ObjectFunctionality.cs
  - ObjectSaver.cs
  - TileManager.cs
- Menu scripts
  - InputFieldController.cs
  - MainMenuController.cs
  - PauseMenuController.cs

- Movement script
  - PlayerMovement.cs
- API and AI python code
  - api_server.py
  - sd3dfast.py
  - sdxlturbo.py

```
//
//WFC Scripts
//
//MapManager script
//
using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;
using UnityEngine.UI;

public class MapManager : MonoBehaviour
{
    private static MapManager instance;
    //grid dimensions can be set
    public int gridWidth;
    public int gridHeight;
    //all rooms have the same size at the minute
    public int cellSize;
    public string mapName;
    //list of all the tile prefabs
    public List<TileData> tiles;
    //2d array to hold the grid
    public GridCell[,] grid;
    public TileManager tileManager;
    private int numberOfRuns = 0;
    public MapSaver mapSaver;
```

```
//directions for the neighbours
public Vector2Int[] directions = new Vector2Int[]
{
    //south, west, north, east
    new Vector2Int(0,-1),
    new Vector2Int(-1,0),
    new Vector2Int(0,1),
    new Vector2Int(1,0)
};

//singleton pattern to ensure only one instance of the WFCManager
public static MapManager MapManagerInstance
{
    get
    {
        return instance;
    }
}
private void Awake()
{
    if (instance != null && instance != this)
    {
        Destroy(this.gameObject);
    }
    else
    {
        instance = this;
    }
}

//Method to run the WFC
public void RunWFC(int width, int height)
{
    //set the grid with the given dimensions
    InitialiseGrid(width, height);
```

```
    Debug.Log("Running WFC");
    //boolean to check if all cells have collapsed
    bool allCollapsed = false;
    //boolean to ensure a valid path exists
    bool pathExists = false;

    while (!allCollapsed)
    {
        //get the cell with the lowest entropy
        GridCell lowestEntropyCell = GetLowestEntropy();
        //collapse the cell
        CollapseCell(lowestEntropyCell);
        //propagate the constraints
        Propagate(lowestEntropyCell);
        //check if all cells have collapsed
        allCollapsed = CheckAllCollapsed();
        //if not repeat
    }
    //check if a path exists
    pathExists = CheckPath();
    if (pathExists)
    {
        Debug.Log("Number of times run: " + numberOfRuns++);
        Debug.Log("Path exists: " + pathExists);
        Debug.Log("Instantiating tiles");
        InstantiateTiles();
    }
    //set number of time you want to run the WFC
    else if (numberOfRuns < 20000)
    {
        Debug.Log("Number of times run: " + numberOfRuns++);
        RunWFC(width, height);
    }
}
//initialise the grid
public void InitialiseGrid(int width, int height)
```

```
{
    //set the grid dimensions
    gridHeight = height;
    gridWidth = width;
    //cell size could be set here but is default set to 10 in the application
    //cellSize = size;

    //create the grid with the given dimensions
    grid = new GridCell[gridWidth, gridHeight];
    //populate the grid with cells that have all possible tiles available
    for (int x = 0; x < gridWidth; x++)
    {
        for (int y = 0; y < gridHeight; y++)
        {
            //pass the location and the list of tiles to the grid cell
            grid[x, y] = new GridCell(new Vector2Int(x, y), tiles);
        }
    }

    Debug.Log("Grid initialised");
    //here we will remove the boundary tiles
    RemoveBoundaryTiles(grid);
}
//Removes the illegal tiles on the boundary of the grid
public void RemoveBoundaryTiles(GridCell[,] grid)
{
    Debug.Log("Removing boundary tiles");

    foreach (GridCell cell in grid)
    {
        List<TileData> validTiles = new List<TileData>();

        foreach (TileData tile in cell.possibleTiles)
        {
            bool isValid = true;
```

```
            // Left boundary (West doors not allowed)
            if (cell.gridPosition.x == 0 && tile.doorPositions[1] == 1)
            {
                isValid = false;
            }

            // Right boundary (East doors not allowed)
            if (cell.gridPosition.x == gridWidth - 1 && tile.doorPositions[3] == 1)
            {
                isValid = false;
            }

            // Bottom boundary (South doors not allowed)
            if (cell.gridPosition.y == 0 && tile.doorPositions[0] == 1)
            {
                isValid = false;
            }

            // Top boundary (North doors not allowed)
            if (cell.gridPosition.y == gridHeight - 1 && tile.doorPositions[2] == 1)
            {
                isValid = false;
            }

            if (isValid)
            {
                validTiles.Add(tile);
            }
        }
        cell.possibleTiles = validTiles;
    }
}
//returns a GridCell object with the lowest entropy
public GridCell GetLowestEntropy()
{
    //we cycle through the grid and get the cell with the lowest amount of possiblities
```

```
    //this will be the cell with the lowest entropy
    GridCell lowestEntropyCell = null;

    // loop through the grid
    foreach (GridCell cell in grid)
    {
        // if the cell has not collapsed
        if (cell.chosenTile == null)
        {
            //if the lowestEntropyCell is null, set it to the current cell
            if (lowestEntropyCell == null)
            {
                lowestEntropyCell = cell;
            }
            //if the current cell has less possible tiles than the lowestEntropyCell
            else if (cell.possibleTiles.Count < lowestEntropyCell.possibleTiles.Count)
            {
                lowestEntropyCell = cell;
            }
        }
    }
    return lowestEntropyCell;
}
public void CollapseCell(GridCell cell)
{
    //if cell is already collapsed, return
    if (cell.chosenTile != null)
    {
        return;
    }
    //if there are no possible tiles left, return
    if (cell.possibleTiles.Count == 0)
    {
        //error handling
        Debug.LogWarning("No possible tiles left for cell at " + cell.gridPosition);
        return;
```

```
    }
    //select a random tile from the possible tiles
    int randomTileIndex = UnityEngine.Random.Range(0, cell.possibleTiles.Count);

    //set the chosen tile to the random tile
    cell.chosenTile = cell.possibleTiles[randomTileIndex];

    //set the possible tiles list to just the chosen tile
    cell.possibleTiles = new List<TileData> { cell.chosenTile };
}
public void Propagate(GridCell cell)
{
    //checks if the cell has collapsed
    if (cell.chosenTile == null)
    {
        return;
    }

    //the cells neighbours in the grid
    foreach (Vector2Int direction in directions)
    {
        //get the neighbour using the direction offset
        Vector2Int neighbourPos = cell.gridPosition + direction;
        //check if the neighbour is within the grid
        if (IsValidPosition(neighbourPos))
        {
            //get the neighbour cell
            GridCell neighbour = grid[neighbourPos.x, neighbourPos.y];

            //if the neighbour hasnt collapsed, ie it has a list to update
            if (neighbour.chosenTile == null)
            {
                int sourceWallIndex = 0;
                int neighbourWallIndex = 0;

                //directions are south, west, north, east in that order
```

```
                    if (direction == directions[2])
                    {
                        //north neighbour
                        sourceWallIndex = 2;
                        neighbourWallIndex = 0;
                    }
                    else if (direction == directions[3])
                    {
                        //east neighbour
                        sourceWallIndex = 3;
                        neighbourWallIndex = 1;
                    }
                    else if (direction == directions[0])
                    {
                        //south neighbour
                        sourceWallIndex = 0;
                        neighbourWallIndex = 2;
                    }
                    else if (direction == directions[1])
                    {
                        //west neighbour
                        sourceWallIndex = 1;
                        neighbourWallIndex = 3;
                    }

                    //filter the possible tiles based on the doors
                    List<TileData> tilesToRemove = new List<TileData>();
                    foreach (TileData tile in neighbour.possibleTiles)
                    {
                        //if the cell has a door in the direction and the neighbour doesnt have a door in
the opposite direction
                        //remove the tile from the list of possible tiles
                        if      (cell.chosenTile.doorPositions[sourceWallIndex]      ==      1      &&
tile.doorPositions[neighbourWallIndex] == 0)
                        {
                            //if the neighbour tile doesnt have a door in the opposite
```

```
                        //add it to the list of tiles to remove
                        tilesToRemove.Add(tile);
                    }
                }
                //remove the tiles from the list of possible tiles
                foreach (TileData tile in tilesToRemove)
                {
                    neighbour.possibleTiles.Remove(tile);
                }
            }
            //recursively call the propagate method on the neighbour
            //removed this as it was causing a stack overflow exception
            // Propagate(neighbour);
        }
    }
}

//checks if the position is within the grid
public bool IsValidPosition(Vector2Int pos)
{
    if (pos.x >= 0 && pos.x < gridWidth && pos.y >= 0 && pos.y < gridHeight)
    {
        return true;
    }
    return false;
}

//check if all cells have collapsed
public bool CheckAllCollapsed()
{
    //loop throught the grid and check if cell.chosenTile is null
    foreach (GridCell cell in grid)
    {
        if (cell.chosenTile == null)
        {
            return false;
```

```
            }
        }
        return true;
    }

    //method to check if there is a path from 0,0 to all other cells, through the connected door positions
    //for example a cell with doorPositions[0,1,0,0] will connect with [0,0,1,0] or [0,0,1,1] or [0,1,1,0]
etc
    //if not then reset the grid and run the WFC again
    //we will use a breadth first search to check if there is a path
    public bool CheckPath()
    {
        //bfs so we need a queue
        Queue<GridCell> queue = new Queue<GridCell>();
        //need a list to store the visited cells
        List<GridCell> visited = new List<GridCell>();
        //start from the 0,0 cell
        queue.Enqueue(grid[0, 0]);
        //add the 0,0 cell to the visited list
        visited.Add(grid[0, 0]);
        //queue all the connected cells and visit them until we reach the end
        while (queue.Count > 0)
        {
            //take the first cell from the queue
            GridCell currentCell = queue.Dequeue();
            //find the connected cells add them to the queue
            for (int i = 0; i < directions.Length; i++)
            {
                if (currentCell.chosenTile.doorPositions[i] == 1)
                {
                    //get the neighbour cell
                    Vector2Int neighbourPos = currentCell.gridPosition + directions[i];
                    //check if the neighbour is within the grid
                    if (IsValidPosition(neighbourPos))
                    {
                        //Debug.Log("the neighbour is valid");
```

```
                        GridCell neighbour = grid[neighbourPos.x, neighbourPos.y];
                        //check if the neighbour has been visited
                        if (!visited.Contains(neighbour) && isDoorConnected(currentCell, neighbour, i))
                        {
                            //add the neighbour to the queue
                            queue.Enqueue(neighbour);
                            //add the neighbour to the visited list
                            visited.Add(neighbour);
                        }
                    }
                }
            }

        }
        //check if all cells have been visited
        if (visited.Count == gridWidth * gridHeight)
        {
            return true;
        }
        else
        {
            Debug.Log("Path does not exist");
            return false;
        }
    }

    public bool isDoorConnected(GridCell cell, GridCell neighbour, int directionIndex)
    {
        //should be the opposite of the direction
        //so index +2
        int neighbourDoorDirectionIndex = (directionIndex + 2) % 4;

        //return true if bnoth have a 1 at teh door position
        return cell.chosenTile.doorPositions[directionIndex] == 1 &&
neighbour.chosenTile.doorPositions[neighbourDoorDirectionIndex] == 1;
```

```
    }

    public void InstantiateTiles()
    {
        //loop through the grid and instantiate the tiles
        for (int x = 0; x < gridWidth; x++)
        {
            for (int y = 0; y < gridHeight; y++)
            {
                //grab the cell
                GridCell cell = grid[x, y];
                //get the chosen tile
                TileData chosenTile = cell.chosenTile;
                //instantiate the tile
                GameObject tile = Instantiate(chosenTile.tilePrefab, new Vector3(x * cellSize, 0, y *
cellSize), chosenTile.tilePrefab.transform.rotation);

                //sets the grid position of the button objects in the tile
                ButtonController[] buttonControllers = tile.GetComponentsInChildren<ButtonController>();
                foreach (ButtonController buttonController in buttonControllers)
                {
                    buttonController.setGrid(x, y);
                }

                //code to load the saved images to the corresponding buttons objects
                for (int i = 0; i < cell.imagePaths.Length; i++)
                {
                    String imagePath = cell.imagePaths[i];
                    //check if the image exists
                    if (!string.IsNullOrEmpty(imagePath) && File.Exists(imagePath))
                    {
                        byte[] imageBytes = File.ReadAllBytes(imagePath);
                        Texture2D tex = new Texture2D(2, 2, TextureFormat.RGBA32, false);
                        //load the image from the byte array
                        if (tex.LoadImage(imageBytes))
                        {
```

```
                        Sprite webSprite = APIManager.APIInstance.SpriteFromTexture2D(tex);

                        if (i < buttonControllers.Length)
                        {
                            //grabs the image component from the button controller through the canvaa
                            Image image = buttonControllers[i].promptCanvasController.image;
                            image.sprite = webSprite;
                        }
                    }
                }
            }
        }
    }
    //lock the cursor and hide it, allow the player to move, sets the menu flag to false
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
    PlayerMovement.PlayerMovementInstance.CanMove = true;
    PauseMenuController.PMCInstance.SetIsPauseMenuOpen(false);
}

public void saveMap()
{
    //create a new mapData object
    MapData mapData = new MapData();
    mapData.height = gridHeight;
    mapData.width = gridWidth;
    mapData.cellSize = cellSize;
    //flatten the 2d array of grid cells into a list
    for (int x = 0; x < gridWidth; x++)
    {
        for (int y = 0; y < gridHeight; y++)
        {
            GridCell cell = grid[x, y];
            //create a new roomData object and load it with the cell data
            RoomData roomData = new RoomData();
            roomData.x = x;
```

```
            roomData.y = y;
            roomData.roomType = cell.chosenTile.tileName;

            //add the image paths
            roomData.imagePaths = cell.imagePaths;
            //add the roomData object to the mapData object
            mapData.rooms.Add(roomData);
        }
    }
    //add the object data to the mapData object
    mapData.objects = ObjectSaver.ObjectSaverInstance.objectDataList;
    //save the mapData object
    mapSaver.SaveGrid(mapData, mapName);
}

public void ReconstructGrid(MapData map)
{
    //rebuilt the grid from the map data
    gridHeight = map.height;
    gridWidth = map.width;
    cellSize = map.cellSize;
    grid = new GridCell[gridWidth, gridHeight];
    //get the tiles from the tile manager
    tiles = tileManager.GetTileDatas();
    //populate the grid with the room data
    foreach (RoomData room in map.rooms)
    {
        //create a new grid cell
        GridCell cell = new GridCell(new Vector2Int(room.x, room.y), tiles);
        //set the chosen tile
        foreach (TileData tile in tiles)
        {
            if (tile.tileName.Equals(room.roomType))
            {
                cell.chosenTile = tile;
            }
```

```
            }
            //set the image paths
            cell.imagePaths = room.imagePaths;
            //add the cell to the grid
            grid[room.x, room.y] = cell;
        }
        //instantiate the tiles
        InstantiateTiles();
    }
}

//
//GridCell script
//
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class GridCell
{
    //reptresent each cell in the grid where a tile/room can be placed
    //keep track of the cells position in the grid
    public Vector2Int gridPosition;
    //a list of possible tiles that can be placed in this cell
    public List<TileData> possibleTiles;
    //the collapsed cell, the chosen TileData
    public TileData chosenTile = null;
    //image paths at the spawers
    public string[] imagePaths = new string[4];

    public GridCell(Vector2Int gridPos, List<TileData> tiles)
    {
        gridPosition = gridPos;
        possibleTiles = new List<TileData>(tiles);
    }
}
```

```
//
//TileData script
//

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class TileData : ScriptableObject
{
    //holds the data of the tile
    public string tileName;
    //position of doors to be stored, can be 1 per wall, so 0 for no door, 1 for
    //door positions will be stored in the order of south, west, north, east
    public List<int> doorPositions = new List<int> { 0, 0, 0, 0 };

    //holds the prefab of the tile
    public GameObject tilePrefab;
}

//
//NewMap Script
//
using System.Collections;
using System.Collections.Generic;
using TMPro;
using Unity.VisualScripting;
using UnityEngine;

public class NewMap : MonoBehaviour
{
    public GameObject newMapPanel;
    public TMP_InputField mapNameInput;
    public PauseMenuController pauseMenuPanel;
    void Start()
```

```
{
    //panel to name the map
    newMapPanel.SetActive(true);
    //initialise the grid with the dimensions from main menu scene
    int mapSizeX = MapSize.mapSizeX;
    int mapSizeY = MapSize.mapSizeY;

    MapManager.MapManagerInstance.RunWFC(mapSizeX, mapSizeY);

    //unlock the mouse and lock the movement
    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
    PlayerMovement.PlayerMovementInstance.CanMove = false;
    //pause menu flag so m doesnt open it
    pauseMenuPanel.SetIsPauseMenuOpen(true);
}

public void OnConfirmButtonClicked()
{
    if (mapNameInput != null)
    {
        //sets the map name
        MapManager.MapManagerInstance.mapName = mapNameInput.text;
        //saves the map under the name
        MapManager.MapManagerInstance.saveMap();
        //closes the panel and hides the mouse
        newMapPanel.SetActive(false);
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
        PlayerMovement.PlayerMovementInstance.CanMove = true;
        //resets the pause menu flag so m can open it
        pauseMenuPanel.SetIsPauseMenuOpen(false);
    }
    else
    {
        Debug.Log("map name input is null");
```

```
        }
    }
}

//
//MapSize Script
//
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;
using UnityEngine.SceneManagement;
//class for setting the map size
public class MapSize : MonoBehaviour
{
    public TMP_InputField mapSizeXInput;
    public TMP_InputField mapSizeYInput;

    //static variables to be accessed across scenes
    public static int mapSizeX;
    public static int mapSizeY;

    //set the map size, called from the button
    public void SetXY()
    {
        if (mapSizeXInput.text != null && mapSizeYInput.text != null)
        {
            mapSizeX = int.Parse(mapSizeXInput.text);
            mapSizeY = int.Parse(mapSizeYInput.text);

            //load the scene
            SceneManager.LoadScene("NewMapTemplate");
        }
        else
        {
            Debug.Log("Map size input is null");
```

```
        }
    }
}
//
//Spawner scripts
//
//APIManager
//

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Networking;
using System.IO;

public class APIManager : MonoBehaviour
{
    private static APIManager instance;
    private string apiEndpoint;
    //to make sure only one api call is made at a time
    public bool isCallingAPI = false;
    private string localPath;
    private int gridx;
    private int gridy;
    private int buttonIndex;

    //singleton pattern for easy access to the APIManager
    public static APIManager APIInstance
    {
        get
        {
            return instance;
        }
    }
    private void Awake()
    {
```

```
    if (instance != null && instance != this)
    {
        Destroy(this.gameObject);
    }
    else
    {
        instance = this;
    }
}
public void GetImageFromAPI(string prompt, int x, int y, int index, System.Action<Sprite> onSuccess)
{
    //set the grid location and button index from where the call is made
    gridx = x;
    gridy = y;
    buttonIndex = index;
    StartCoroutine(GetImage(prompt, onSuccess));
}
private IEnumerator GetImage(string prompt, System.Action<Sprite> onSuccess)
{
    //api endpoint
    apiEndpoint = "http://localhost:5000/generateImage";
    //only 1 call at a time
    if (isCallingAPI)
    {
        Debug.Log("API call already in progress");
        yield break;
    }
    //sets flag
    isCallingAPI = true;

    //form to send the prompt
    WWWForm form = new WWWForm();
    form.AddField("prompt", prompt);
    //using the unity web request
    using (UnityWebRequest request = UnityWebRequest.Post(apiEndpoint, form))
    {
```

```
            //start the request
            yield return request.SendWebRequest();

            //error handling
            if (request.result != UnityWebRequest.Result.Success)
            {
                Debug.LogError("Error fetching image: " + request.error);
                Debug.Log("Server response: " + request.downloadHandler.text);
            }
            else
            {
                //get the image bytes as a byte array
                byte[] result = request.downloadHandler.data;
                //dummy texture
                Texture2D tex = new Texture2D(2, 2, TextureFormat.RGBA32, false);
                //load the image from the byte array
                //this seems to work better than the DownloadHandlerTexture.GetContent(request)
                bool loaded = tex.LoadImage(result);

                if (loaded)
                {
                    //convert the texture to a sprite
                    Sprite webSprite = SpriteFromTexture2D(tex);
                    //store the image file in the persistent data path
                    string savedImagePath = ImageSaver.ImageSaverInstance.SaveImage(result, prompt);
                    //saves teh image path to the specific room/grid cell for the specific button
                    MapManager.MapManagerInstance.grid[gridx,      gridy].imagePaths[buttonIndex]      =
savedImagePath;
                    //MapManager.MapManagerInstance.saveMap();

                    //return the sprite to the callback function
                    onSuccess?.Invoke(webSprite);
                }
                else
                {
                    Debug.Log("Error loading image");
```

```
            }
        }
    }
    //reset the flag
    isCallingAPI = false;
}

public string Get3DObjectFromAPI(string prompt, System.Action<string> onSuccess)
{
    StartCoroutine(GetObject(prompt, onSuccess));
    localPath += prompt + ".glb";
    //return the path to the object in the persistent data path
    //this will be used by the object loader to instantiate the object
    return localPath;
}
private IEnumerator GetObject(string prompt, System.Action<string> onSuccess)
{
    //only 1 call at a time
    if (isCallingAPI)
    {
        Debug.Log("API call already in progress");
        yield break;
    }
    //set flag
    isCallingAPI = true;
    //api endpoint
    apiEndpoint = "http://localhost:5000/generate3D";

    //form to send the prompt
    WWWForm form = new WWWForm();
    form.AddField("prompt", prompt);
    //using the unity web request
    using (UnityWebRequest webRequest = UnityWebRequest.Post(apiEndpoint, form))
    {
        //start  request
        yield return webRequest.SendWebRequest();
```

```
            if (webRequest.result == UnityWebRequest.Result.Success)
            {
                //file name in storage is the prompt
                string filename = prompt + ".glb";
                //ensure the directory structure exists
                localPath = Path.Combine(Application.persistentDataPath, "Saves", "Objects");
                string dir = Path.GetDirectoryName(localPath);
                if (!Directory.Exists(dir))
                {
                    Directory.CreateDirectory(dir);
                }
                //file path
                string filePath = Path.Combine(localPath, filename);
                //write the downloaded bytes to file
                File.WriteAllBytes(filePath, webRequest.downloadHandler.data);
                Debug.Log("GLB file saved to: " + localPath);
                //return the path to the object
                onSuccess?.Invoke(localPath);
            }
            else
            {
                //error handling
                Debug.LogError($"Error downloading GLB: {webRequest.error}");
            }
        }
        //reset flag
        isCallingAPI = false;
    }

    //helper to convert the texture to a sprite
    public Sprite SpriteFromTexture2D(Texture2D texture)
    {
        //convert the texture to a sprite
        return Sprite.Create(texture, new Rect(0.0f, 0.0f, texture.width, texture.height), new
Vector2(0.5f, 0.5f), 100.0f);
```

```
    }
}


//
//ButtonController script
//

using System.Collections;
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.UI;

public class ButtonController : MonoBehaviour
{
    public float interactionDistance = 2.0f;
    public int buttonIndex;
    private int gridx;
    private int gridy;
    private Transform player;
    public Transform buttonTransform;
    public PromptCanvasController promptCanvasController;
    public bool isButtonPressed = false;
    public TMP_Dropdown dropdown;
    public GameObject dropdownCanvas;

    // Start is called before the first frame update
    void Start()
    {
        player = Camera.main.transform;
    }

    private void OnMouseDown()
    {
```

```
        //player must be within the set distance, pause menu must be closed, and there cannot be any api
call inprogress
        if (Vector3.Distance(Camera.main.transform.position, transform.position) <= interactionDistance &&
!PauseMenuController.PMCInstance.GetIsPauseMenuOpen() && !APIManager.APIInstance.isCallingAPI)
        {
            //pause menu flag to true, gets reset on cancel or submit
            PauseMenuController.PMCInstance.SetIsPauseMenuOpen(true);

            //lock player movement and show the cursor
            Cursor.lockState = CursorLockMode.None;
            Cursor.visible = true;
            PlayerMovement.PlayerMovementInstance.CanMove = false;
            //show the selection dropdown
            dropdownCanvas.SetActive(true);
            //for selecting the option, 2d or 3d
            dropdown.onValueChanged.AddListener(delegate { DropdownValueChanged(dropdown); });
            //button animation
            if (!isButtonPressed)
            {
                //lower teh button and set the flag
                Vector3 newPosition = buttonTransform.position;
                newPosition.y -= 0.05f;
                buttonTransform.position = newPosition;
                isButtonPressed = true;
            }
        }

    }
    //calls corresponding method to the dropdown value
    private void DropdownValueChanged(TMP_Dropdown change)
    {
        if (change.value == 1)
        {
            Option2D();
        }
        else if (change.value == 2)
```

```
    {
        Option3D();
    }
}
//2d image option
public void Option2D()
{
    //the type is 1 for 2d image
    promptCanvasController.type = 1;
    //opens the prompt canvas
    promptCanvasController.gameObject.SetActive(true);
    //gives the button info to know from which spawner the prompt is made
    promptCanvasController.setButtonInfo(gridx, gridy, buttonIndex);
    //closes the dropdown
    dropdownCanvas.SetActive(false);
}
public void Option3D()
{
    //the type is 2 for 3d object
    promptCanvasController.type = 2;
    //opens the prompt canvas
    promptCanvasController.gameObject.SetActive(true);
    //closes the dropdown
    dropdownCanvas.SetActive(false);
}
//logs the room coordinates for the button, from the mapmanager
public void setGrid(int x, int y)
{
    gridx = x;
    gridy = y;
}
//dont think i used these in the end, but good to have
public int getGridX()
{
    return gridx;
}
```

```
    public int getGridY()
    {
        return gridy;
    }
    //helper for the button animation, will be red while call in progress
    public void SetButtonRed()
    {
        GetComponent<MeshRenderer>().material.color = Color.red;
    }
    //helper for the button animation, will be green when call is done
    public void SetButtonGreen()
    {
        GetComponent<MeshRenderer>().material.color = Color.green;
    }
}
//
//ObjectLoader script
//
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using GLTFast;
using System.IO;

public class ObjectLoader : MonoBehaviour
{
    // make a singleton
    public static ObjectLoader instance;
    public static ObjectLoader ObjectLoaderInstance
    {
        get
        {
            return instance;
        }
    }
```

```
private void Awake()
{
    //singleton pattern to ensure only one instance of the APIManager
    if (instance != null && instance != this)
    {
        Destroy(this.gameObject);
    }
    else
    {
        instance = this;
    }
}
public async void LoadObject(string path, string prompt)
{
    //load the object from the path using the gltf importer
    var temp = new GltfImport();
    bool success = await temp.Load(path);

    if (success)
    {
        Debug.Log("GLB loaded");
        GameObject parent = new GameObject("ImportedGLB");
        //set the name of the object to the prompt for easy identification
        parent.name = prompt;
        await temp.InstantiateMainSceneAsync(parent.transform);
        //add the functionality to the object
        var draggable = parent.AddComponent<ObjectFunctionality>();

        Vector3 position = PlayerMovement.PlayerMovementInstance.transform.position;
        //spawns overhead
        position.y += 1.5f;
        parent.transform.position = position;
        //add a box collider
        BoxCollider boxCollider = parent.AddComponent<BoxCollider>();
        RecalculateBoundsFromChildren(parent, boxCollider);
        //update the object data structure for saving
```

```
            ObjectSaver.ObjectSaverInstance.UpdateData(prompt,                parent.transform.position,
parent.transform.rotation, parent.transform.localScale);
        }
        else
        {
            //error handle
            Debug.LogError("Failed to load GLB");
        }
    }

    private void RecalculateBoundsFromChildren(GameObject parent, BoxCollider boxCollider)
    {
        var meshRenderers = parent.GetComponentsInChildren<MeshRenderer>();

        //start with the first renderer's bounds
        Bounds combinedBounds = meshRenderers[0].bounds;
        //encapsulate all child mesh bounds
        for (int i = 1; i < meshRenderers.Length; i++)
        {
            combinedBounds.Encapsulate(meshRenderers[i].bounds);
        }
        //BoxCollider center is in local space, so convert from world to local
        boxCollider.center = parent.transform.InverseTransformPoint(combinedBounds.center);
        boxCollider.size = combinedBounds.size;
    }

    //object loader from json
    public async void LoadObjectFromSave(ObjectData objectData)
    {
        //load the object from the json data
        string localPath = Path.Combine(Application.persistentDataPath, "Saves", "Objects");
        string filePath = Path.Combine(localPath, objectData.name + ".glb");
        var temp = new GltfImport();
        bool success = await temp.Load(filePath);
        if (success)
        {
```

```
            //create the object
            GameObject parent = new GameObject("ImportedGLB");
            //set the name of the object to the json name
            parent.name = objectData.name;
            Debug.Log("Parent name: " + parent.name);
            //instantiate the object
            await temp.InstantiateMainSceneAsync(parent.transform);
            //add the draggable script
            var draggable = parent.AddComponent<ObjectFunctionality>();
            //set the object to the parameters in json
            parent.transform.position = objectData.position;
            parent.transform.rotation = objectData.rotation;
            parent.transform.localScale = objectData.scale;
            //add a box collider
            BoxCollider boxCollider = parent.AddComponent<BoxCollider>();
            RecalculateBoundsFromChildren(parent, boxCollider);
            ObjectSaver.ObjectSaverInstance.UpdateData(objectData.name,                objectData.position,
objectData.rotation, objectData.scale);
        }
    }
}
//
//PromptCanvasController
//
using System.Collections;
using System.Collections.Generic;
using System.IO;
using TMPro;
using UnityEngine;
using UnityEngine.UI;


public class PromptCanvasController : MonoBehaviour
{

    public TMP_InputField promptInput;
```

```
    public GameObject promptCanvas;
    public Image image;
    public ButtonController ButtonController;
    private int gridx;
    private int gridy;
    private int buttonIndex;
    public int type;

    //for setting the button location on the grid and in the room, used for saving the image and loading
it back in the correct location
    public void setButtonInfo(int x, int y, int index)
    {
        gridx = x;
        gridy = y;
        buttonIndex = index;
    }
    public void OnConfirm()
    {
        //grabs the input text
        string prompt = promptInput.text;
        //type 1 is 2d image
        if (type == 1)
        {
            //reset the pause menu flag on confirm
            PauseMenuController.PMCInstance.SetIsPauseMenuOpen(false);

            //get the image from the API
            //prompt cant be empty and no api call in progress
            if (!prompt.Equals("") && !APIManager.APIInstance.isCallingAPI)
            {
                //button animation colour
                ButtonController.SetButtonRed();
                //call the api manager to get the image
                APIManager.APIInstance.GetImageFromAPI(prompt, gridx, gridy, buttonIndex, (Sprite result)
=>
                {
```

```
            //set the image to the returned sprite
            image.sprite = result;
            //button animation colour green
            ButtonController.SetButtonGreen();
            //button animation, pops up again
            buttonAnimation();
        });
    }
    else
    {
        //button animation, pops up again
        buttonAnimation();
        //print error
        Debug.Log("Prompt is empty");
    }

    //close the prompt canvas
    promptCanvas.SetActive(false);
    //hide cursor again and player can move
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;
    PlayerMovement.PlayerMovementInstance.CanMove = true;
}
//type 2 is 3d object
else if (type == 2)
{
    //reset the pause menu flag on confirm
    PauseMenuController.PMCInstance.SetIsPauseMenuOpen(false);

    //code for 3d prompt to api controller
    //cant be empty and no api call in progress
    if (!prompt.Equals("") && !APIManager.APIInstance.isCallingAPI)
    {
        //button animation colour
        ButtonController.SetButtonRed();
        //call the api manager to get the 3d object
```

```
            //will return a path to the object
            //object loader will instantiate the glb file with the gltf plugin
            APIManager.APIInstance.Get3DObjectFromAPI(prompt, (string localPath) =>
            {
                //prompt is the file name
                string filePath = Path.Combine(localPath, prompt);
                filePath = filePath + ".glb";
                //load the 3d object
                ObjectLoader.ObjectLoaderInstance.LoadObject(filePath, prompt);
                //button animation colour green
                ButtonController.SetButtonGreen();
                //button animation, pops up again
                buttonAnimation();
            });
        }
        else
        {
            buttonAnimation();
            Debug.Log("Prompt is empty");
        }
        //close the prompt canvas
        promptCanvas.SetActive(false);
        //hide cursor again and player can move
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
        PlayerMovement.PlayerMovementInstance.CanMove = true;
    }
}
public void OnCancel()
{
    //reset the pause menu flag on cancel
    PauseMenuController.PMCInstance.SetIsPauseMenuOpen(false);
    //button animation colour
    ButtonController.SetButtonGreen();
    //close the prompt canvas
    promptCanvas.SetActive(false);
```

```
        //hide cursor again and player can move
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
        PlayerMovement.PlayerMovementInstance.CanMove = true;
        //button animation, pops up again
        buttonAnimation();
    }
    //helper for the button animations
    public void buttonAnimation()
    {
        //if the button flag is set
        if (ButtonController.isButtonPressed)
        {
            //raise the button back up and reset the flag
            Vector3 newPosition = ButtonController.buttonTransform.position;
            newPosition.y += 0.05f;
            ButtonController.buttonTransform.position = newPosition;
            ButtonController.isButtonPressed = false;
        }
    }
}
//
//Save-Load
//
//ImageSaver script
//
using System.Collections;
using System.Collections.Generic;
using System.IO;
using UnityEngine;

//class to save the images to a file in the persistent data path
public class ImageSaver : MonoBehaviour
{
    private static ImageSaver instance;
    public static ImageSaver ImageSaverInstance
```

```
{
    get
    {
        return instance;
    }
}
private void Awake()
{
    if (instance != null && instance != this)
    {
        Destroy(this.gameObject);
    }
    else
    {
        instance = this;
    }
}

//method to save the image to a file
public string SaveImage(byte[] imageBytes, string file)
{
    //Images to be saved in a folder called Images in the Saves folder
    string imageDirectory = Path.Combine(Application.persistentDataPath, "Saves", "Images");

    //check if the directory exists
    if (!Directory.Exists(imageDirectory))
    {
        //if it doesn't exist create it
        Directory.CreateDirectory(imageDirectory);
    }

    //create teh file path string
    string fileName = file + ".png";
    string filePath = Path.Combine(imageDirectory, fileName);

    //check if the file already exists in the saves folder we will append a number
```

```
        int counter = 1;
        while (File.Exists(filePath))
        {
            //if it does add counter to the file name
            fileName = file + counter + ".png";
            //log the name to the console
            Debug.Log("New file name: " + fileName);
            //create the new file path
            filePath = Path.Combine(imageDirectory, fileName);
            //increment the counter
            counter++;
        }

        //write the image to the file
        File.WriteAllBytes(filePath, imageBytes);
        Debug.Log("Image saved to: " + filePath);

        //return the file path to be used in the save file
        return filePath;
    }
}

//
//LoadMenuController script
//
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using System.IO;

//class to load the map data from a file in the persistent data path
public class LoadMenuController : MonoBehaviour
{
    public MapLoader mapLoader;
    public GameObject loadButtonPrefab;
```

```
public Transform contentParent;

private void Start()
{
    //sets the pause menu boolean to true so the menu wont appear when m is pressed
    PauseMenuController.PMCInstance.SetIsPauseMenuOpen(true);
    FillList();
}
//method to fill the list of maps
private void FillList()
{
    //get the list of maps from the map loader
    string[] files = mapLoader.GetMaps();
    if (files == null)
    {
        Debug.Log("No Maps found");
        return;
    }
    //cycle through the list of maps
    foreach (string file in files)
    {
        //get the name of the map cutting the .json extension
        string fileName = Path.GetFileNameWithoutExtension(file);
        //create a button for each file
        GameObject loadButton = Instantiate(loadButtonPrefab, contentParent);

        //set the text of the button to the name of the file
        loadButton.GetComponentInChildren<TMPro.TextMeshProUGUI>().text = fileName;

        //add a listener to the button
        Button button = loadButton.GetComponent<Button>();
        if (button != null)
        {
            //if the button is clicked call the load map function with teh filename
            button.onClick.AddListener(() => mapLoader.LoadMap(fileName));
        }
```

```
        }
    }
}

//
//MapData script
//
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

[Serializable]
public class RoomData
{
    //store an x and y position in the grid
    public int x;
    public int y;
    //store the roomtype
    public string roomType;
    //Array of the images saves in the rorom
    //order bottom left, top left, top right, bottom right
    public string[] imagePaths = new string[4];
}
[Serializable]
public class ObjectData
{
    //store the name of the 3Dobject
    public string name;
    //store the position, rotation and scale of the object
    public Vector3 position;
    public Quaternion rotation;
    public Vector3 scale;
}
[Serializable]
public class MapData
```

```
{
    //store the dimensions of the grid
    public int width;
    public int height;
    public int cellSize;

    //data structure to store the room data objects
    public List<RoomData> rooms = new List<RoomData>();
    //data structure to store the 3d object data
    public List<ObjectData> objects = new List<ObjectData>();
}
//
//MapLoader script
//
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System.IO;

//class to load the map data from a file in the persistent data path
public class MapLoader : MonoBehaviour
{
    //reference to the load menu canvas
    public GameObject LoadMenuCanvas;
    //lock the player movement while choosing a map to load
    private void Start()
    {
        Cursor.lockState = CursorLockMode.None;
        Cursor.visible = true;
        PlayerMovement.PlayerMovementInstance.CanMove = false;
    }

    public void LoadMap(string fileName)
    {
        //disable the load menu
        LoadMenuCanvas.SetActive(false);
```

```
    //call the load grid function
    MapData map = LoadGrid(fileName);
    //set the map name
    MapManager.MapManagerInstance.mapName = fileName;
    //reconstruct the grid from the map data loaded
    MapManager.MapManagerInstance.ReconstructGrid(map);
    //cycle through the objects from the map and instantiate them
    foreach (ObjectData objectData in map.objects)
    {
        //instantiate the object
        ObjectLoader.ObjectLoaderInstance.LoadObjectFromSave(objectData);
    }
}

//helper method to load the map data from a file
public MapData LoadGrid(string fileName)
{
    //get the path to the file from the filename passed from the load menu
    string path = Path.Combine(Application.persistentDataPath, "Saves", fileName + ".json");

    //check if the file exists or not
    if (!File.Exists(path))
    {
        Debug.LogError("File " + fileName + " does not exist");
        return null;
    }

    //read the json from the file
    string json = File.ReadAllText(path);

    //deserialize the json into a MapData object
    MapData map = JsonUtility.FromJson<MapData>(json);
    //return the map object
    return map;
}
```

```
    //method to get the list of maps in the save directory
    //used to populate the load menu
    public string[] GetMaps()
    {
        string saveDirectory = Path.Combine(Application.persistentDataPath, "Saves");
        //if the directory does not exist create it and return null to the load menu list
        if (!Directory.Exists(saveDirectory))
        {
            Debug.LogError("Directory " + saveDirectory + " does not exist");
            Directory.CreateDirectory(saveDirectory);
            return null;
        }
        //get all the files in the directory with the .json extension
        string[] maps = Directory.GetFiles(saveDirectory, "*.json");
        return maps;
    }
}
//
//MapSaver script
//
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System.IO;
//class to save the map data to a file in the persistent data path
public class MapSaver : MonoBehaviour
{
    //string to hold the directory path
    public string directory;
    //function to save the map data to a file
    public void SaveGrid(MapData map, string fileName)
    {
        //convert the map data to a json string
        string json = JsonUtility.ToJson(map);
        //create the directory path
        directory = Path.Combine(Application.persistentDataPath, "Saves");
```

```
        //check if the directory exists
        if (!Directory.Exists(directory))
        {
            //if it doesn't exist create it
            Directory.CreateDirectory(directory);
        }
        //create the file path
        string path = Path.Combine(directory, fileName + ".json");

        //write the json to the file
        File.WriteAllText(path, json);

        //confirm in console that the file has been saved
        Debug.Log("File " + fileName + " saved to " + path);
    }
}

//
//ObjectFunctionality script
//
using UnityEngine;

public class ObjectFunctionality : MonoBehaviour
{
    private Camera mainCamera;
    private bool isHolding;
    private Vector3 offset;
    private float zDistanceToCamera;

    void Start()
    {
        mainCamera = Camera.main;
    }

    void OnMouseDown()
    {
```

```
        isHolding = true;

        //distance from the camera to the object
        zDistanceToCamera = Vector3.Distance(transform.position, mainCamera.transform.position);

        //cnvert the object's position to screen space
        Vector3 screenPos = mainCamera.WorldToScreenPoint(transform.position);

        //mouse position in screen space
        Vector3 mouseScreenPos = new Vector3(Input.mousePosition.x, Input.mousePosition.y, screenPos.z);
        Vector3 mouseWorldPos = mainCamera.ScreenToWorldPoint(mouseScreenPos);

        //calculate offset so the object doesn't snap to the cursor
        offset = transform.position - mouseWorldPos;
    }

    void OnMouseDrag()
    {
        if (!isHolding)
        {
            return;
        }

        //current mouse position in screen space, at the same Z distance
        Vector3    mouseScreenPos    =    new    Vector3(Input.mousePosition.x,    Input.mousePosition.y,
zDistanceToCamera);
        Vector3 mouseWorldPos = mainCamera.ScreenToWorldPoint(mouseScreenPos);

        //Move the object to follow the mouse, maintaining the offset
        transform.position = mouseWorldPos + offset;

        //rotate the object
        if (Input.GetKey(KeyCode.R))
        {
            transform.Rotate(Vector3.up, 0.5f);
        }
```

```
        //scale the object
        //get the scroll wheel input
        float scroll = Input.GetAxis("Mouse ScrollWheel");
        if (Mathf.Abs(scroll) > 0.01f)
        {
            //increase or decrease scale uniformly
            Vector3 newScale = transform.localScale + Vector3.one * scroll * 0.5f;
            //prevent negative or zero scale
            newScale = Vector3.Max(newScale, Vector3.one * 0.01f);
            //set the new scale
            transform.localScale = newScale;
        }

        //testing deleting the object
        if (Input.GetKeyDown(KeyCode.Delete))
        {
            //removes from storage
            ObjectSaver.ObjectSaverInstance.DeleteObject(gameObject.name);
            //removes from scene
            Destroy(gameObject);
        }
    }

    void OnMouseUp()
    {
        isHolding = false;
        //on mouse up log the position, rotation adn scale of the object to the object saver data structure
        ObjectSaver.ObjectSaverInstance.UpdateData(gameObject.name,                    transform.position,
transform.rotation, transform.localScale);
    }
}

//
//ObjectSaver script
//
```

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class ObjectSaver : MonoBehaviour
{
    //object sver singleton
    //singleton pattern to ensure only one instance of the object saver
    //useful for calling the instance from other scripts
    private static ObjectSaver instance;

    //list of the object data
    //name, position, rotation, scale
    public List<ObjectData> objectDataList = new List<ObjectData>();
    public static ObjectSaver ObjectSaverInstance
    {
        get
        {
            return instance;
        }
    }
    private void Awake()
    {
        //singleton pattern to ensure only one instance of the obejct saver
        if (instance != null && instance != this)
        {
            Destroy(this.gameObject);
        }
        else
        {
            instance = this;
        }
    }

    //method to add the object data to the data structure
```

```
public void UpdateData(string name, Vector3 position, Quaternion rotation, Vector3 scale)
{
    //check if the object is already in the list
    for (int i = 0; i < objectDataList.Count; i++)
    {
        if (objectDataList[i].name.Equals(name))
        {
            //if it is update the data
            objectDataList[i].position = position;
            objectDataList[i].rotation = rotation;
            objectDataList[i].scale = scale;
            return;
        }
    }
    //if not create
    ObjectData objectData = new ObjectData();
    objectData.name = name;
    objectData.position = position;
    objectData.rotation = rotation;
    objectData.scale = scale;
    objectDataList.Add(objectData);
}
//function to remove an object from the list
public void DeleteObject(string name)
{
    //cycle through the list to find the object
    for (int i = 0; i < objectDataList.Count; i++)
    {
        //if the object is found remove it
        if (objectDataList[i].name.Equals(name))
        {
            objectDataList.RemoveAt(i);
            break;
        }
    }
    //remove it from the persistent data if it exists
```

```
        string path = Application.persistentDataPath + "/Saves/Objects/" + name + ".glb";

        if (System.IO.File.Exists(path))
        {
            Debug.Log("Deleting file " + path);
            System.IO.File.Delete(path);
        }
    }
}

//
//TileManager script
//

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

//clss to hold all the tile prefabs
public class TileManager : MonoBehaviour
{
    //list of all the tile prefabs
    public List<TileData> tiles;
    //return the list of tile prefabs
    public List<TileData> GetTileDatas()
    {
        return tiles;
    }
}
//
//Menu Scripts
//
//InputFieldController script
//
using System.Collections;
using System.Collections.Generic;
```

```
using UnityEngine;

public class InputFieldController : MonoBehaviour
{
    public GameObject inputPanel;
    //open the input panel
    public void open()
    {
        inputPanel.SetActive(true);
    }
    //close the input panel
    public void close()
    {
        inputPanel.SetActive(false);
    }
}

//
//MainMenuController script
//

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class MainMenuContoller : MonoBehaviour
{
    public GameObject ChooseOptionPanel;
    public GameObject MapSizePanel;

    public void Start()
    {
        MapSizePanel.SetActive(false);
    }
    public void NewMap()
```

```
    {
        ChooseOptionPanel.SetActive(false);
        MapSizePanel.SetActive(true);
    }
    public void LoadMap()
    {
        SceneManager.LoadScene("LoadMapTemplate");
    }
}

//
//PauseMenuController script
//

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;
using UnityEngine.SceneManagement;

public class PauseMenuController : MonoBehaviour
{
    public GameObject pauseMenu;
    public GameObject saveMenuPanel;

    private bool isPauseMenuOpen = false;
    private static PauseMenuController instance;

    public static PauseMenuController PMCInstance
    {
        get
        {
            return instance;
        }
    }
```

```
private void Awake()
{
    //singleton pattern to ensure only one instance of the WFCManager
    if (instance != null && instance != this)
    {
        Destroy(this.gameObject);
    }
    else
    {
        instance = this;
    }
}

// Start is called before the first frame update
void Start()
{
    pauseMenu.SetActive(false);
    saveMenuPanel.SetActive(false);
}

void Update()
{
    //check for m key to bring up menu
    if (Input.GetKeyDown(KeyCode.M) && !isPauseMenuOpen)
    {
        Debug.Log("M key pressed");
        isPauseMenuOpen = true;
        pauseMenu.SetActive(true);
        TogglePauseMenu();
    }
}

public void TogglePauseMenu()
{
    //lock movement
    if (isPauseMenuOpen)
```

```
    {
        Cursor.lockState = CursorLockMode.None;
        Cursor.visible = true;
        PlayerMovement.PlayerMovementInstance.CanMove = false;
    }
    else
    {
        //unlock movement
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
        PlayerMovement.PlayerMovementInstance.CanMove = true;
    }
}

public void OnResumeButtonClicked()
{
    isPauseMenuOpen = false;
    pauseMenu.SetActive(false);
    TogglePauseMenu();
}
public void OnSaveButtonClicked()
{
    saveMenuPanel.SetActive(true);
    pauseMenu.SetActive(false);
}

public void OnMainMenuButtonClicked()
{
    SceneManager.LoadScene("MainMenu");
}

public void OnQuitButtonClicked()
{
    //not working in editor
    //would work in a build
    Application.Quit();
```

```
    }
    public void OnConfirmButtonClicked()
    {
        //save map
        MapManager.MapManagerInstance.saveMap();
        saveMenuPanel.SetActive(false);
        pauseMenu.SetActive(true);
    }

    public void OnCancelButtonClicked()
    {
        saveMenuPanel.SetActive(false);
        pauseMenu.SetActive(true);
    }
    public void SetIsPauseMenuOpen(bool value)
    {
        isPauseMenuOpen = value;
    }
    public bool GetIsPauseMenuOpen()
    {
        return isPauseMenuOpen;
    }
}

//
//Movement Script
//
//PlayerMovement
//
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    private static PlayerMovement instance;
```

```
public Camera playerCamera;
public float walkSpeed = 4f;
public float lookSpeed = 2f;
public float lookXLimit = 45f;
public float height = 1.5f;
private Vector3 moveDirection = Vector3.zero;
private float rotationX = 0;
private CharacterController characterController;
private bool canMove = true;

public static PlayerMovement PlayerMovementInstance
{
    get
    {
        return instance;
    }
}
private void Awake()
{
    if (instance != null && instance != this)
    {
        Destroy(this.gameObject);
    }
    else
    {
        instance = this;
    }
}

public bool CanMove
{
    get
    {
        return canMove;
    }
```

```
        set
        {
            canMove = value;
        }
    }

    void Start()
    {
        //lock and hide the cursor, get the character controller
        characterController = GetComponent<CharacterController>();
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }
    void Update()
    {
        //get the forward and right vectors
        Vector3 forward = transform.TransformDirection(Vector3.forward);
        Vector3 right = transform.TransformDirection(Vector3.right);
        //speed of movement
        float curSpeedX = canMove ? (walkSpeed) * Input.GetAxis("Vertical") : 0;
        float curSpeedY = canMove ? (walkSpeed) * Input.GetAxis("Horizontal") : 0;
        //move the player
        moveDirection = (forward * curSpeedX) + (right * curSpeedY);
        characterController.Move(moveDirection * Time.deltaTime);
        if (canMove)
        {
            rotationX += -Input.GetAxis("Mouse Y") * lookSpeed;
            rotationX = Mathf.Clamp(rotationX, -lookXLimit, lookXLimit);
            playerCamera.transform.localRotation = Quaternion.Euler(rotationX, 0, 0);
            transform.rotation *= Quaternion.Euler(0, Input.GetAxis("Mouse X") * lookSpeed, 0);
        }
    }
}
#
#API and AI python code
#
```

```
#api_server
#
from flask import Flask, request, send_file
import io
from PIL import Image
import sdxlturbo as sd
import sd3dfast as sf3d

app = Flask(__name__)

@app.route('/generateImage', methods=['POST'])
def generateImage():
    # get the data from the request
    data = request.form.get('prompt')

    # generate the image and store as an image object
    image = sd.generate_image(data)
    print("Image generated")

    # creates a byte stream to store the image
    img_io = io.BytesIO()
    # save the image to the byte stream
    image.save(img_io, format='PNG')
    # set the position of the byte stream to the beginning
    img_io.seek(0)
    # return the image as a png file
    return send_file(img_io, mimetype='image/png')

@app.route("/generate3D", methods=['POST'])
def generate3D():
    # get teh data from the request´
    data = request.form.get('prompt')

    # first generate the image
    image = sd.generate_image(data)
    image_path = "temp_generated_image.png"
```

```
    image.save(image_path)
    print("Image generated")

    print("Generating 3D model")
    # give the image to the stable-fast-3d model
    glbPath = sf3d.generate_glb(image_path, data)
    print("3D model generated")
    #glbPath is the file path for the glb file that was generated
    print(glbPath)
    # return the glb
    return send_file(glbPath, mimetype='model/gltf-binary', as_attachment=True, download_name='model.glb')

if __name__ == "__main__":
    app.run(port=5000, debug=True)

#
#sd3dfast
#
import os
import re
from contextlib import nullcontext

import torch
import rembg
from PIL import Image
from tqdm import tqdm

from sf3d.system import SF3D
from sf3d.utils import get_device, remove_background, resize_foreground

# Taken from stable-fast-3d/run.py
def generate_glb(
    image_path,                     # Image path
    device=None,                         # Device to use ('cuda', 'mps', or 'cpu'); if None, determined
automatically
    pretrained_model="stabilityai/stable-fast-3d",  # Pretrained model identifier or local path
```

```
    foreground_ratio=0.85,          # Ratio to resize the foreground, not changed from original file from
gh
    output_dir="output/",           # Directory to save output files, will change this later
    texture_resolution=1024,        # Texture atlas resolution
    remesh_option="none",           # Remeshing option: "none", "triangle", or "quad"
    target_vertex_count=-1,         # Target vertex count (-1 means no reduction)
    batch_size=1                    # Batch size for inference
):
    #
    # Determine device if not provided
    if device is None:
        device = get_device()
    if not (torch.cuda.is_available() or torch.backends.mps.is_available()):
        device = "cpu"
    print("Device used:", device)
    os.makedirs(output_dir, exist_ok=True)

    # Create a rembg session and process the image
    rembg_session = rembg.new_session()
    im = Image.open(image_path).convert("RGBA")
    im = remove_background(im, rembg_session)
    im = resize_foreground(im, foreground_ratio)

    # Save the processed image temporarily in a subfolder
    temp_folder = os.path.join(output_dir, "temp")
    os.makedirs(temp_folder, exist_ok=True)
    temp_image_path = os.path.join(temp_folder, "inputImage.png")
    im.save(temp_image_path)

    images = [im]

    # Load the stable-fast-3d model
    model = SF3D.from_pretrained(
        pretrained_model,
        config_name="config.yaml",
        weight_name="model.safetensors",
```

```
    )
    model.to(device)
    model.eval()

 # Process the image and generate the 3D model (GLB file)
    for i in tqdm(range(0, len(images), batch_size)):
        batch = images[i : i + batch_size]
        if torch.cuda.is_available():
            torch.cuda.reset_peak_memory_stats()
        with torch.no_grad():
            ctx = torch.autocast(device_type=device, dtype=torch.bfloat16) if "cuda" in device else
nullcontext()
            with ctx:
                mesh, _ = model.run_image(
                    batch,
                    bake_resolution=texture_resolution,
                    remesh=remesh_option,
                    vertex_count=target_vertex_count,
                )
        # Create an output filename based on the prompt
        base_filename = "3DObject"
        glb_filename = base_filename + ".glb"
        glb_filepath = os.path.join(output_dir, glb_filename)

        # Save the generated GLB file
        if len(batch) == 1:
            mesh.export(glb_filepath, include_normals=True)
        else:
            for j, m in enumerate(mesh):
                out_path = os.path.join(output_dir, f"{base_filename}_{j}.glb")
                m.export(out_path, include_normals=True)

    #remove the temporary image
    os.remove(temp_image_path)
    #return the path and the api can return the file
    return glb_filepath
```

```
#
#sdxlturbo
#
import torch
import torch_directml
from diffusers import AutoPipelineForText2Image

#wrap this to return an image when called
def generate_image(prompt):
    # Create a DirectML device
    dml = torch_directml.device()

    print(prompt)
    # Load the pipeline
    # Use float16 instead of bfloat16 for AMD + DirectML

    # Move the pipeline to the DirectML device

    pipe = AutoPipelineForText2Image.from_pretrained("stabilityai/sdxl-turbo", torch_dtype=torch.float16,
variant="fp16")
    pipe.to(dml)

    image = pipe(prompt=prompt, num_inference_steps=3, guidance_scale=0.0).images[0]

    print("returning image")
    return image
```

Appendix 4 Screen shots of the project implementation